

Testing and Debugging in Python

Unit Testing, Assertions, Debugging,
and PDB Tool

What is Testing?

- ▶ Testing is the process of evaluating a program or application to ensure it works as expected.
- ▶ It helps identify bugs, errors, and any unexpected behavior in the code.
- ▶ Types of Testing:
 1. Unit Testing: Testing individual components or functions.
 2. Integration Testing: Testing interactions between integrated components.
 3. System Testing: Testing the complete system as a whole.
 4. Acceptance Testing: Testing the system from the user's perspective.

Unit Testing in Python

- ▶ Unit Testing is a type of testing where individual units (functions or methods) are tested in isolation.
- ▶ Python's `unittest` module provides tools for creating and running unit tests.
- ▶ Syntax:

```
import unittest
```

```
class TestClass(unittest.TestCase):  
    def test_method(self):  
        self.assertEqual(result, expected)
```

```
if __name__ == '__main__':  
    unittest.main()
```

Assertions in Unit Testing

- ▶ Assertions are used to check the expected outcomes during testing. Common assertions:
 - `assertEqual(a, b)`: Verifies `a == b`
 - `assertNotEqual(a, b)`: Verifies `a != b`
 - `assertTrue(a)`: Verifies `a` is `True`
 - `assertFalse(a)`: Verifies `a` is `False`
 - `assertIsNone(a)`: Verifies `a` is `None`
 - `assertIsInstance(a, b)`: Verifies `a` is an instance of class `b`

Debugging in Python

- ▶ Debugging is the process of identifying and fixing bugs or issues in code. It helps to:
 - Inspect code execution
 - Identify why code isn't working as expected
 - Fix errors at runtime
- ▶ Python provides the ``pdb`` tool for interactive debugging.

PDB Tool in Python

- ▶ ``pdb`` (Python Debugger) is a built-in interactive tool that allows you to debug your Python programs.
- ▶ Key commands in ``pdb``:
 - ``n`` (next): Execute the next line of code.
 - ``s`` (step): Step into a function call.
 - ``c`` (continue): Continue execution until the next breakpoint.
 - ``q`` (quit): Exit the debugger.
 - ``p <variable>``: Print the value of a variable.
 - ``l`` (list): List the code around the current line.
 - ``where`` (w): Show the stack trace.
 - ``h``: Display help on commands.